Data Visualization with Seaborn

The subject of this project is Seaborn. It is a Python data visualization library based on Matplotlib. The contents of this project is divided into various sections which are listed below:-

Table of Contents

1. Introduction to Seaborn

Seaborn is a data visualization library in Python. It is used to produce statistical graphics in Python. It is built on top of Matplotlib. It also supports NumPy and Pandas data structures. It also supports statistical units from SciPy.

In the world of analytics, data visualization plays an important role to explore and understand the data. Seaborn is developed to make this process of exploring and understanding the data smooth. It provides a high-level interface to produce high quality statistical graphics. The customizability and backend options for Matplotlib makes it easy to generate publication-quality figures.

In this project, I discuss Seaborn and present a high level overview of various plotting functions and tools associated with Seaborn.

2. Comparison of Seaborn and Matplotlib

Seaborn uses Matplotlib to draw plots. Many actions can be accomplished with only Seaborn functions. Further customization options might require using Matplotlib directly. Seaborn is not an alternative to Matplotlib. We can think of it as a complement to Matplotlib. As it is built on top of Matplotlib, we can call Matplotlib functions directly for creating simple plots.

It is summarized that –

“If Matplotlib tries to make easy things easy and hard things possible, Seaborn tries to make a well-defined set of hard things easy too in a well-defined way.”

Seaborn helps to resolve two major problems faced by Matplotlib. The problems are:-

1. Working with Matplotlib’s default parameters
2. Working with data frames

3. Seaborn API Overview

Seaborn has various types of plots which can satisfy various types of data visualization requirements. The plots can be categorized into seven categories – relational plots, categorical plots, distribution plots, regression plots, matrix plots and multi-plot grids.

4. Seaborn functionality

Seaborn offers lot of functionality which makes it effective for many data visualization tasks. These functionalities are listed below:-

• Seaborn provides a dataset-oriented API to examine relationships between variables.

• It provides functions to fit and visualize linear regression models for different types of independent and dependent variables.

• Seaborn provide functions for visualizing univariate and bivariate distributions and for comparing them between subsets of data.

• It provides plotting functions for using categorical variables to show observations or aggregate statistics.

• It helps us to visualize matrices of data and use clustering algorithms to discover structure in those matrices.

• Seaborn provides a plotting function to plot statistical time series data. The function provides flexible estimation and representation of uncertainty around the estimate.

• It provide tools for choosing styles, colour palettes, palette widgets and utility functions. These tools help us to make beautiful plots that reveal patterns in the data.

• It provides several built-in themes for producing stylish looking Matplotlib graphics.

5. Seaborn installation

To install the latest release of Seaborn, we can use **pip** and run the following command in the terminal:

```
pip install seaborn
```

It is also possible to install the released version using **conda** as follows:

```
conda install seaborn
```

Alternatively, we can use **pip** to install the development version directly from github:

```
pip install git+https://github.com/mwaskom/seaborn.git
```

Dependencies

• Seaborn requires Python 2.7 or 3.5+ platform.

Mandatory dependencies

Seaborn requires following mandatory Python packages to be installed on the system.

• NumPy(>= 1.9.3)

• Scipy (>= 0.14.0)

• Matplotlib (>= 1.4.3)

• Pandas (>= 0.15.2)

Recommended dependencies

Seaborn also needs statsmodels (>= 0.5.0) package to be installed on the system.

6. Import Python libraries

In this section, we import the necessary Python libraries Numpy, Pandas and Matplotlib with their usual shorthand notation.

In [156…
```
# Import the dependencies
# Ignore warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
%matplotlib inline
```

7. Import Seaborn

We can import Seaborn as

```
import seaborn
```

The shorthand notation of Seaborn is **sns**. So, we can import seaborn with usual shorthand notation.

```
import seaborn as sns
```

In [157…
```
# Import seaborn

import seaborn as sns
```

8. Import datasets

Seaborn has few important datasets in the library. When Seaborn is installed, the datasets download automatically. We can download any dataset with **load_dataset()** function.

Importing data as Pandas dataframe

In this section, we will import a dataset. By default, this dataset loads as Pandas dataframe. There is a dataset called **tips** in the library. We can import this dataset with the following line of code:-

In [158…
```
tips = sns.load_dataset('tips')
```

We can view the first five lines of code with the following command:-

In [159…
```
print(tips.head())
```

```
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
```

In [196…
```
tips.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype   
---  ------      --------------  -----   
 0   total_bill  244 non-null    float64 
 1   tip         244 non-null    float64 
 2   sex         244 non-null    category
 3   smoker      244 non-null    category
 4   day         244 non-null    category
 5   time        244 non-null    category
 6   size        244 non-null    int64   
dtypes: category(4), float64(2), int64(1)
memory usage: 7.4 KB
```

To view all the available datasets in the Seaborn library, we can use the following command with the **get_dataset_names()** function as shown below:-

In [160…
```
print(sns.get_dataset_names())
```

```
['anagrams', 'anscombe', 'attention', 'brain_networks', 'car_crashes', 'diamonds', 'dots', 'exercise', 'flights', 'fmri', 'gammas', 'geyser', 'iris', 'mpg', 'penguins', 'planets', 'taxis', 'tips', 'titanic']
```

9. Set aesthetic parameters with set() method

We can set the aesthetic parameters of Seaborn plots with the **set()** method. The aesthetic parameters are context, style, palette, font, font_scale, color_codes, dictionary of rc parameters.

So, we can set the default aesthetic parameters by calling Seaborn's **set()** method as follows:-

In [161…
```
# Set the plot styles with set() method
sns.set_style("white")
sns.set()
```

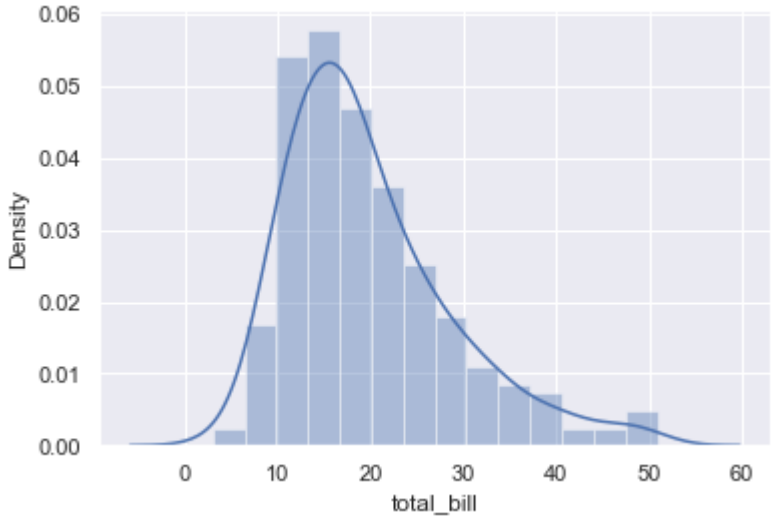
10. Plotting Univariate Distribution with distplot()

The most important thing to do while analysing the data is to understand its distribution. Seaborn helps us to understand the univariate distribution of data.

The **distplot()** function provides a quick look at univariate distribution. This function will plot a histogram that fits the kernel density estimate of the data.

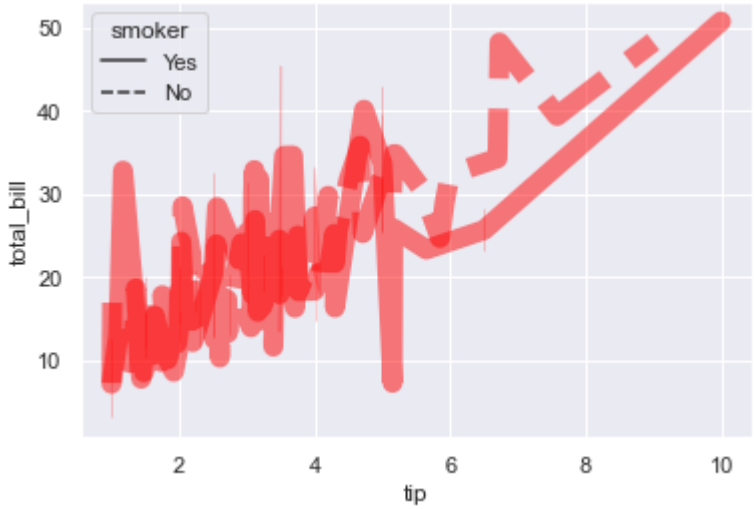
We can use the **distplot()** function as follows:-

In [162…
```
sns.distplot(tips['total_bill'])
plt.show()
```



In [163…
```
sns.lineplot(data=tips,x='tip', y='total_bill', style='smoker', lw=10, co
```

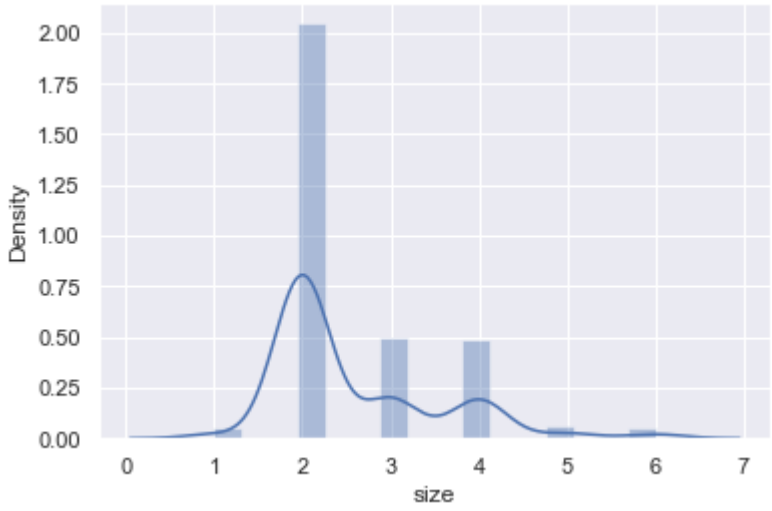
Out[163]:
```
<AxesSubplot:xlabel='tip', ylabel='total_bill'>
```



In [164…
```
tips.columns
```

Out[164]:
```
Index(['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size'], dty
pe='object')
```

In [165…
```
sns.distplot(tips['size'])
plt.show()
```



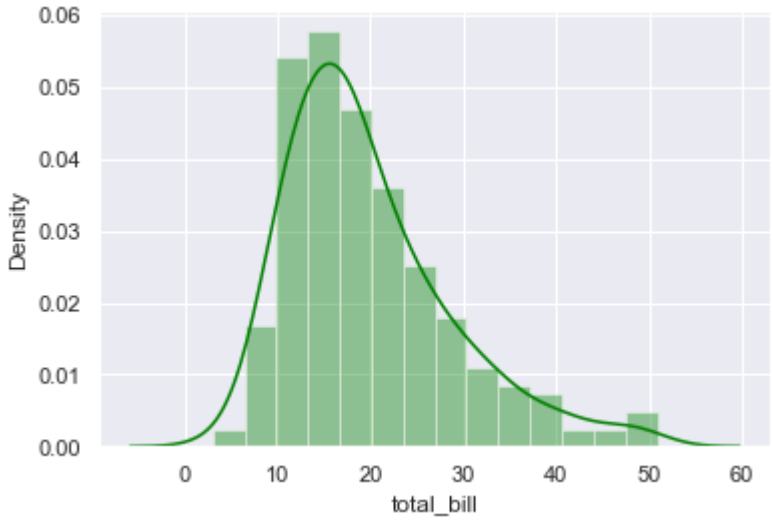
11. Seaborn Histogram

Histograms represent the data distribution by forming bins along the range of the data and then drawing bars to show the number of observations that fall in each bin.

We can use the same **distplot()** function to plot a histogram as follows:-

In [166…
```
sns.distplot(tips['total_bill'], color='green', kde=True)

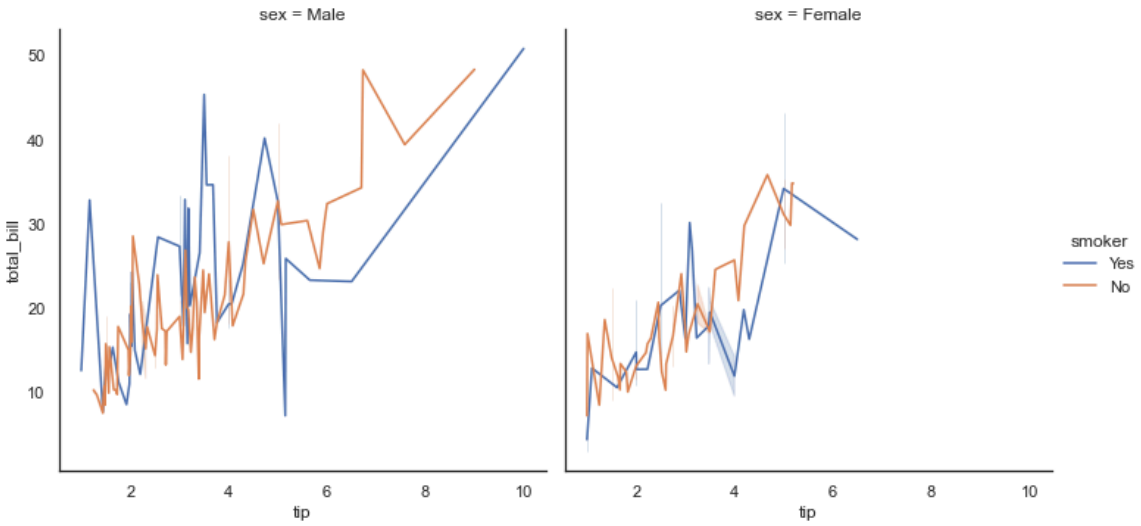
plt.show()
```



In [167…
```
#The major difference between relplot and scatterplot (as well as others)
#To do this, you will use the col or row keyword argument.
#Either of these arguments will take the name of a DataFrame column (seri
#and then create a new column or row (respectively) containing plots, for
#relational plots
#kind=line,scatter
#sns.set(style ="ticks")
```

In [168…
```
#In seaborn, the hue parameter determines which column in the data frame
sns.set_style('white')
sns.relplot(x='tip',y='total_bill', data=tips, kind='line' , hue='smoker'
```

Out[168]:
```
<seaborn.axisgrid.FacetGrid at 0x25c9a5945e0>
```



The kde parameter is set to false. As a result, the representation of the kernel estimation plot will be removed and only histogram is plotted.

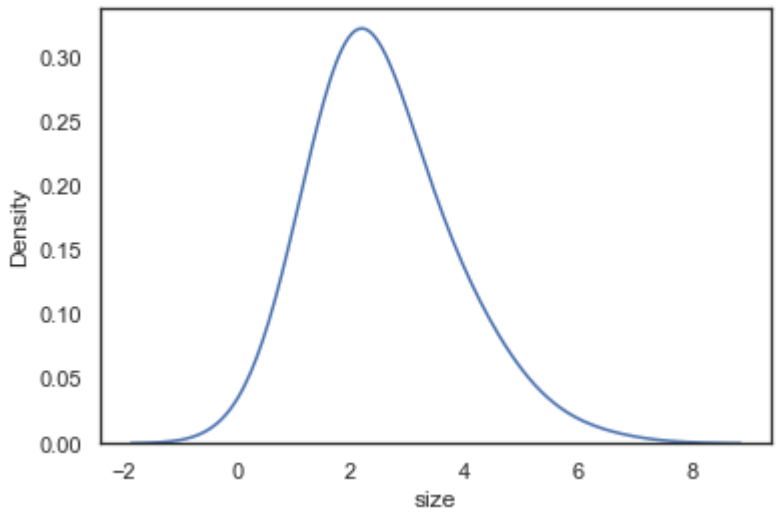
12. Seaborn – Kernel Density Estimates

Kernel Density Estimation (KDE) is a way to estimate the probability density function of a continuous random variable. It is used for non-parametric analysis.

Setting the hist parameter to false in distplot() function will yield the kernel density estimation plot.

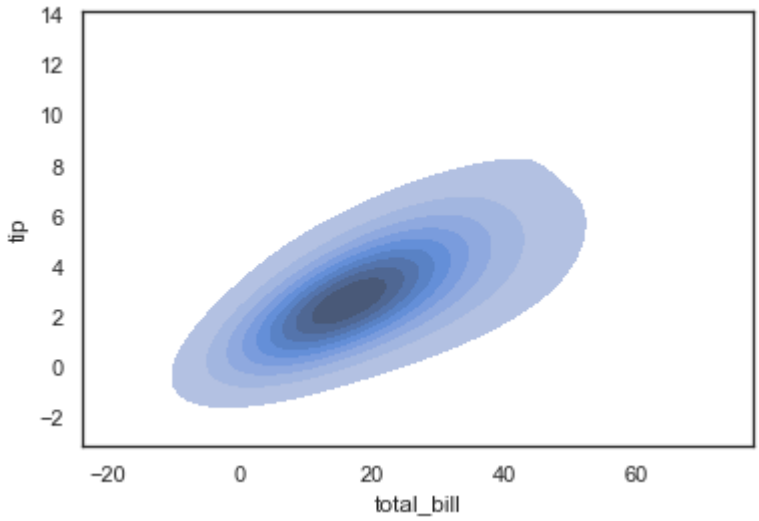
In [169…
```
#sns.distplot(tips['total_bill'])
#shade, Bw,
sns.kdeplot(tips['size'], shade=False, bw=1)

plt.show()
```



In [170…
```
sns.kdeplot(tips['total_bill'],tips['tip'], shade=True, bw=1)

plt.show()
```



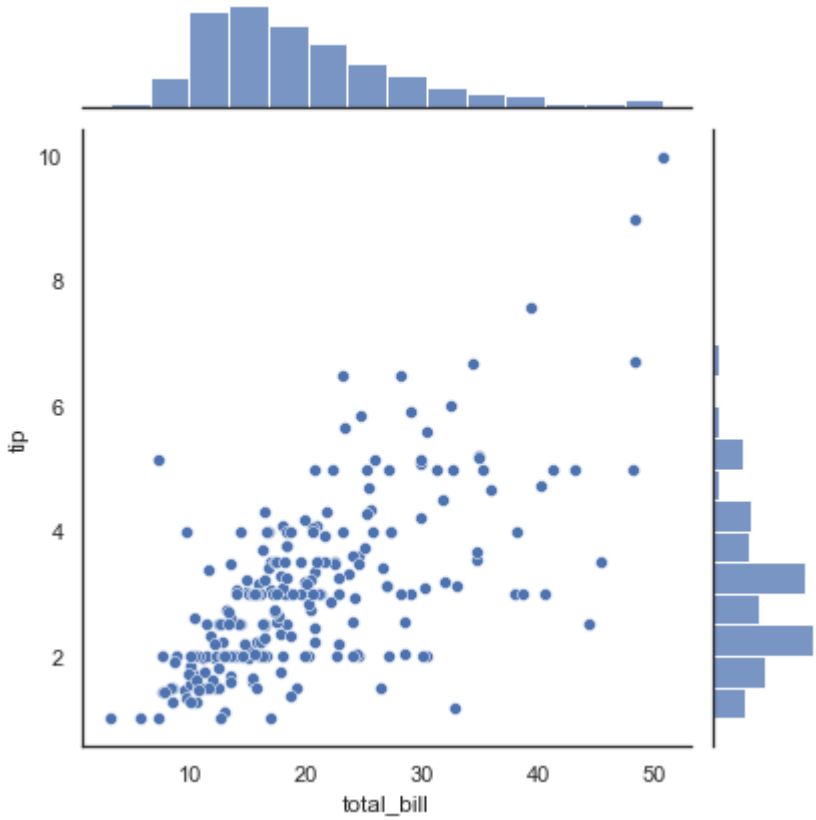
13. Plotting Bivariate Distribution with jointplot()

Bivariate Distribution is used to determine the relation between two variables. This mainly deals with relationship between two variables and how one variable is behaving with respect to the other.

The best way to analyze Bivariate Distribution in seaborn is by using the **jointplot()** function.

Jointplot() creates a multi-panel figure that projects the bivariate relationship between two variables and also the univariate distribution of each variable on separate axes.

In [171…
```
sns.jointplot(x="total_bill", y="tip", data=tips)
plt.show()
```



14. Seaborn - Scatter plot

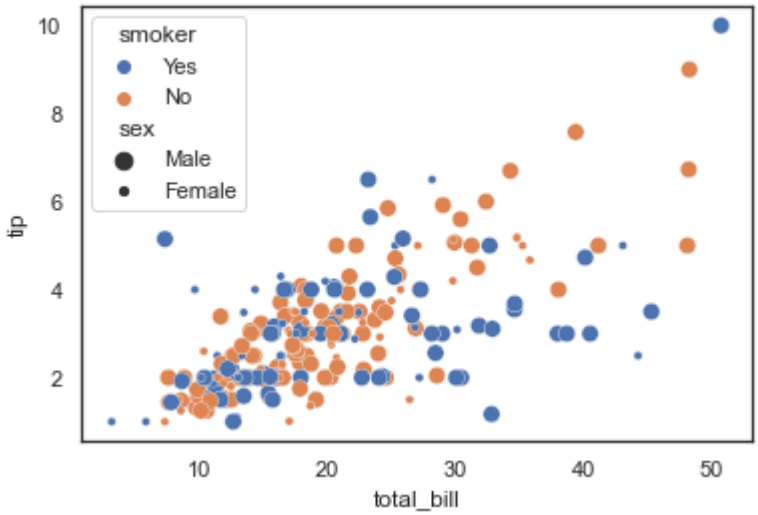
A scatter plot can be used to demonstrate relationship between two variables x and y. A simple scatter plot can be drawn as follows:-

In [172…

```
#hue='smoker', size="sex"

sns.scatterplot(x="total_bill", y="tip", data=tips, size="sex",hue='smoke

plt.show()
```



The relationship between the variables can be shown for different subsets of the data using the `hue` , `size` and `style` parameters.

16. Visualizing pairwise relationship with pairplot()

Some datasets contain many variables. In such cases, the relationship between each and every variable should be analyzed. So, we need to plot pairwise relationships in a dataset.

To plot multiple pairwise bivariate distributions, we can use **pairplot()** function. This shows the relationship for (n,2) combination of variable in a dataframe as a matrix of plots and the diagonal plots are the univariate plots.

By default, this function will create a grid of Axes such that each numeric variable in data will by shared across the y-axes across a single row and the x-axes across a single column

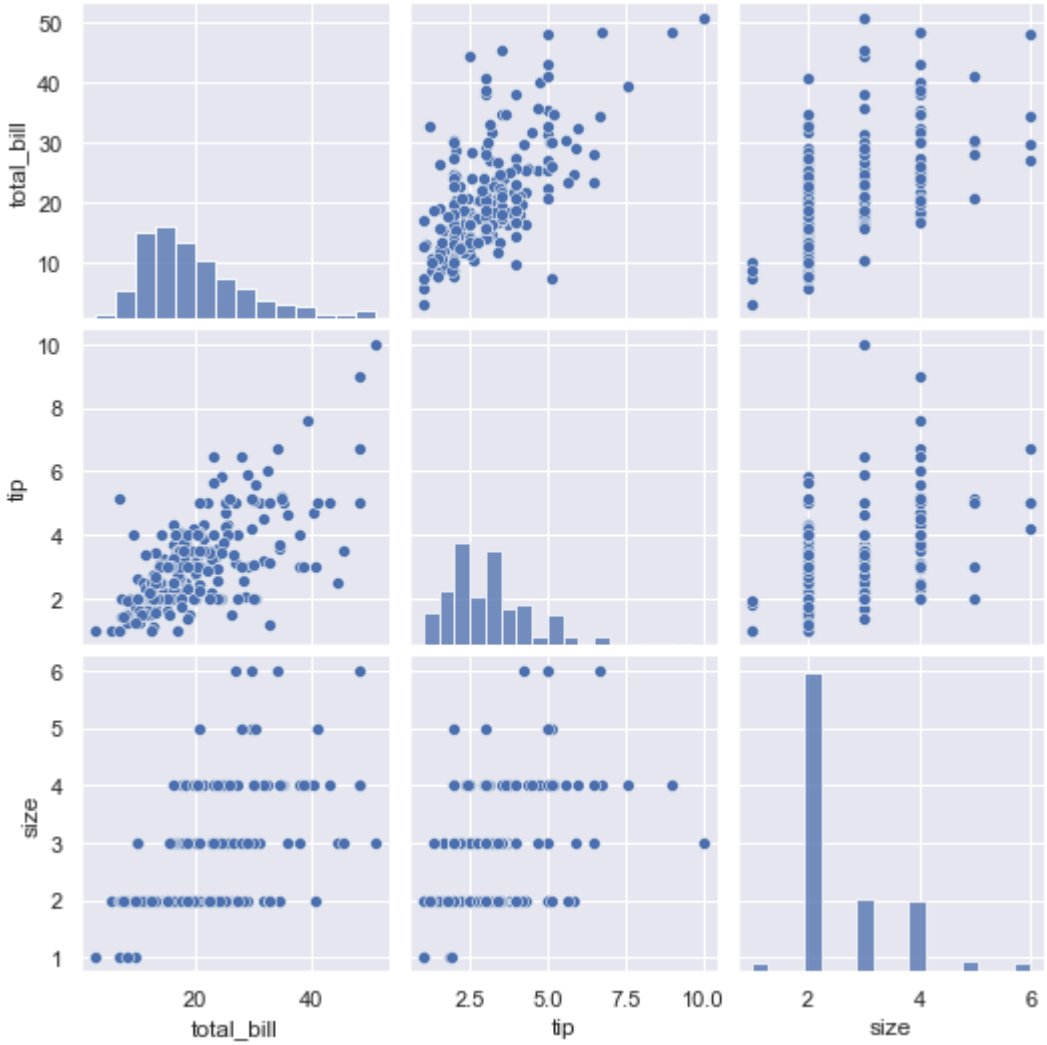
We can plot a pairplot as as follows:-

In [272…

```
#diag_kind='kde' to change the plots on the diagnol and to change the off
# hue can also be set to day, size, sex etc.
sns.set_style("darkgrid")

sns.pairplot(tips)

plt.show()
```

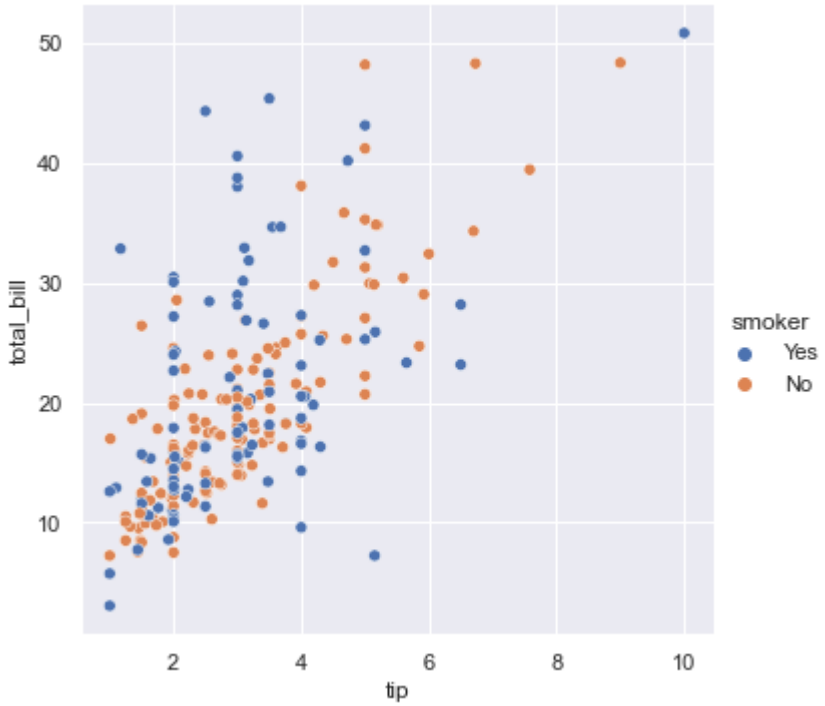


In [223…
```
tips.dtypes
```

Out[223]:
```
total_bill      float64
tip             float64
sex            category
smoker         category
day            category
time           category
size              int64
dtype: object
```

In [273…
```
#relplot()
#scatterplot()
#lineplot()
sns.relplot(data= tips,x='tip',y='total_bill',hue='smoker')
```

Out[273]:
```
<seaborn.axisgrid.FacetGrid at 0x25c9d335700>
```



17. Plotting categorical data

So far, I have covered histogram, scatter plot and kde plots. They are used to analyze the continuous variables under study. These plots are not suitable when the variable under study is categorical.

When one or both the variables under study are categorical, we can use plots like striplot() and swarmplot() to plot categorical data.

18. Seaborn – Strip Plot

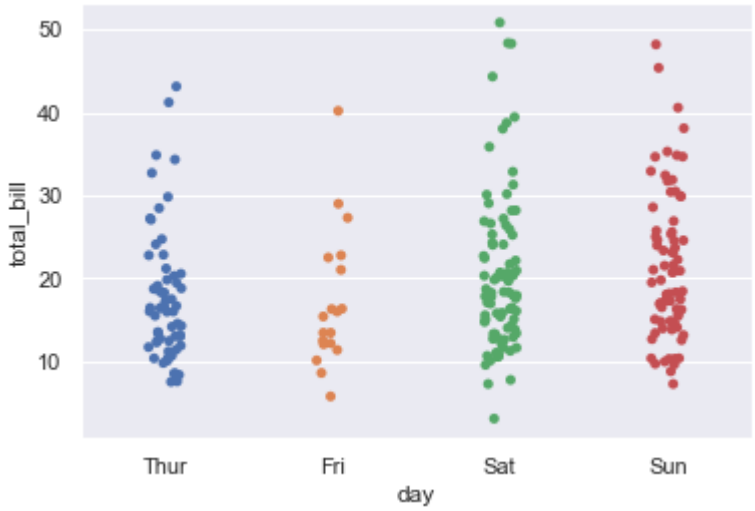
A **stripplot()** is used to draw a scatterplot where one variable is categorical. It represents the data in sorted order along any one of the axis.

We can plot a stripplot as follows:-

In [276…

```
sns.stripplot(x="day", y="total_bill", data=tips)

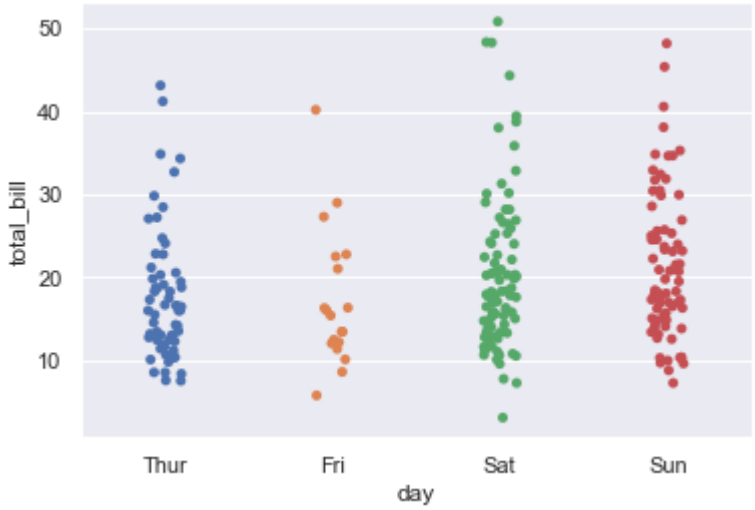
plt.show()
```



We can add jitter to bring out the distribution of values as follows:-

In [282…
```
sns.stripplot(x="day", y="total_bill", data=tips, jitter=True)

plt.show()
```



19. Seaborn – Swarm Plot

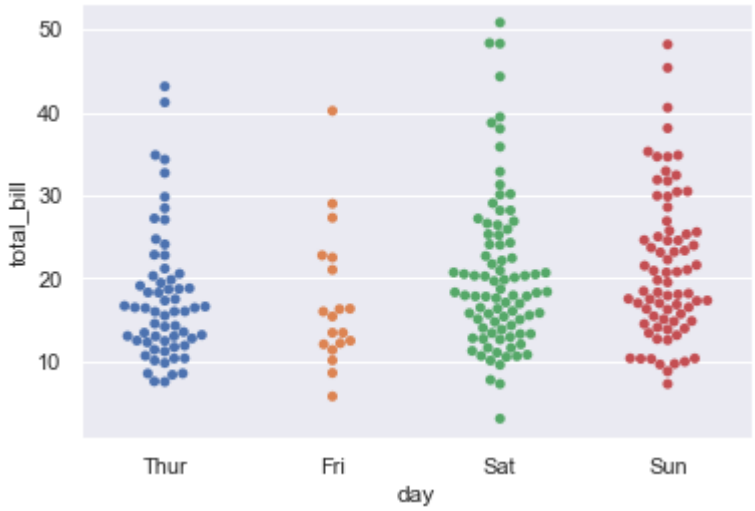
Another option which can be used as an alternate to 'jitter' is to use function **swarmplot()**. This function positions each point of scatter plot on the categorical axis and thereby avoids overlapping points.

So **swarmplot()** can be used to draw categorical scatterplot with non-overlapping points.

We can plot a swarmplot as follows:-

In [288…
```
sns.swarmplot(x="day", y="total_bill", data=tips)

plt.show()
```



20. Distribution of Observations

When dealing with a dataset, the first thing that we want to do is get a sense for how the variables are distributed.

We can use box plots and violin plots to plot the data distribution.

21. Seaborn – Box Plot

We can draw a box plot to show distributions with respect to categories. Boxplot is a convenient way to visualize the distribution of data through their quartiles.

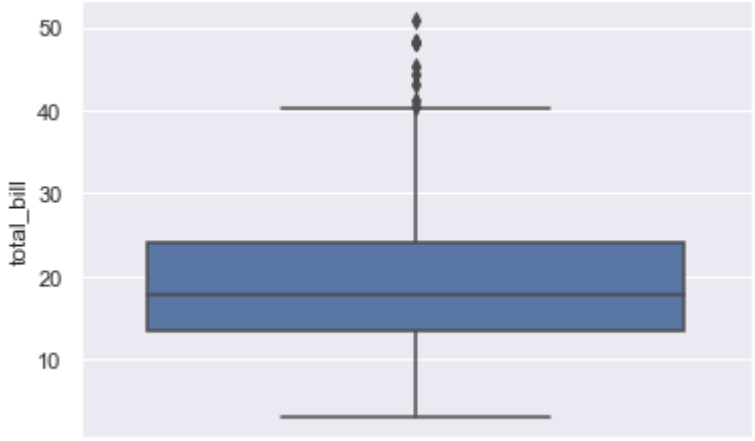
Box plots usually have vertical lines extending from the boxes which are termed as whiskers. These whiskers indicate variability outside the upper and lower quartiles, hence Box Plots are also termed as box-and-whisker plot. Any Outliers in the data are plotted as individual points.

We can draw a single horizontal box-plot as follows:-

In [197…

```
sns.boxplot(y=tips["total_bill"])

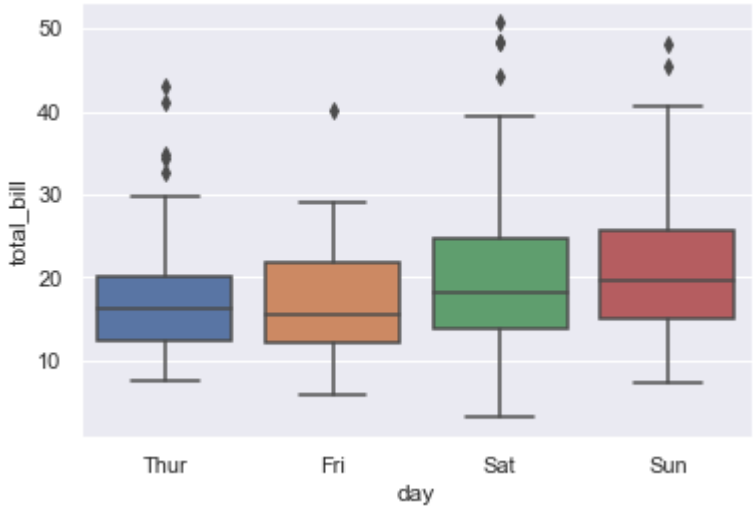
plt.show()
```



We can draw a vertical boxplot grouped by a categorical variable as follows:-

In [198…
```
sns.boxplot(x="day", y="total_bill", data=tips)

plt.show()
```



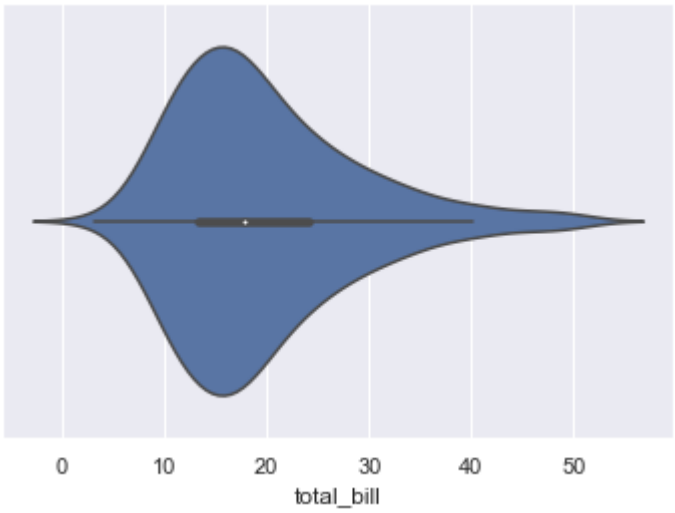
22. Seaborn – Violin Plot

Violin Plots are combinations of the box plot with the kernel density estimates. So, these plots are easier to analyze and understand the distribution of the data.

We can draw a single horizontal violin plot as follows:-

In [290…
```
sns.violinplot(x=tips["total_bill"])

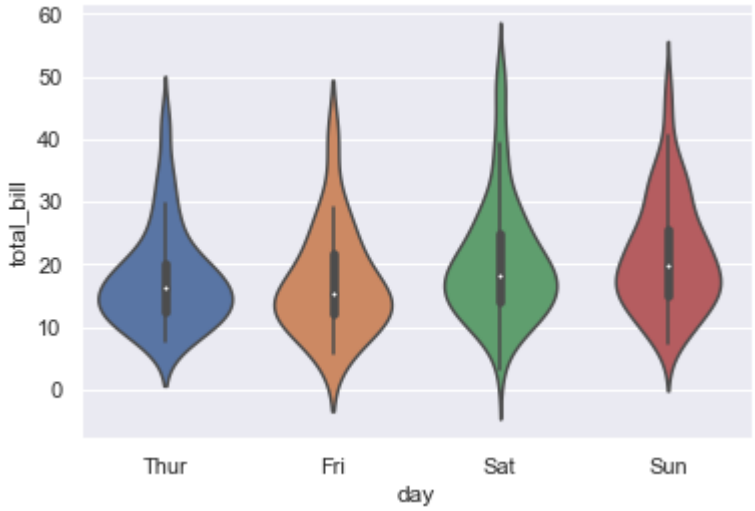
plt.show()
```



We can draw a vertical violinplot grouped by a categorical variable as follows:-

In [335…
```
# hue='sex', split=True, inner='quartiles'(25th, 50th,75th quartiles), co
# scaling based on the count: scale='count'
sns.violinplot(x="day", y="total_bill", data=tips)

plt.show()
```



In [314…
```
# counts the number of male and female in the tips dataset
tips['sex'].value_counts()
```

Out[314]:
```
Male      157
Female     87
Name: sex, dtype: int64
```

23. Statistical estimation with Seaborn

Sometimes, we need to estimate central tendency of a distribution. Mean and median are the very often used techniques to estimate the central tendency of the distribution.

We can use **bar plot** and **point plot** to measure the central tendency of a distribution.

24. Seaborn – Bar Plot

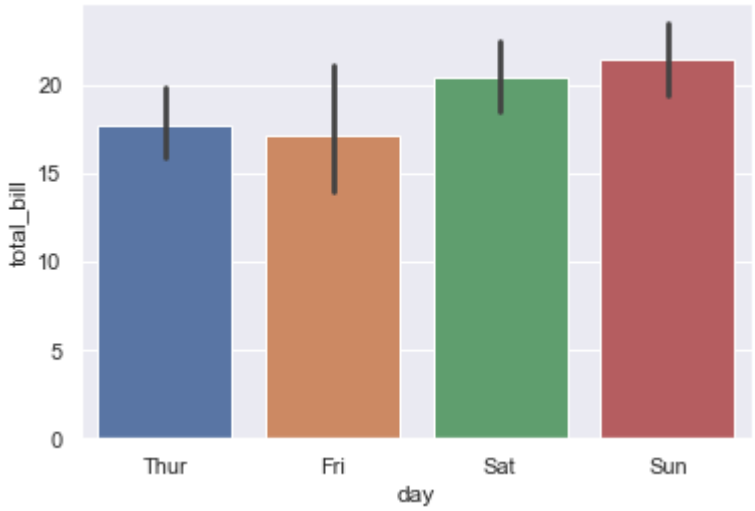
A **barplot** shows the relation between a categorical variable and a continuous variable. The data is represented in rectangular bars where the length the bar represents the proportion of the data in that category.

A barplot show point estimates and confidence intervals as rectangular bars. So, it represents the estimate of central tendency.

We can draw a set of vertical bar plot grouped by a categorical variable as follows:-

In [182…
```
sns.barplot(x="day", y="total_bill", data=tips)

plt.show()
```



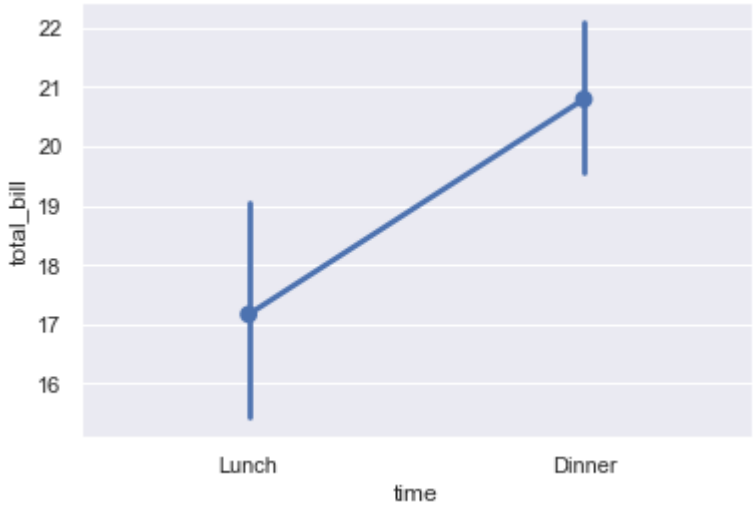
25. Seaborn - Point Plot

Point plots serve same as bar plots but in a different style. Rather than the full bar, the value of the estimate is represented by the point at a certain height on the other axis. The Point plot show point estimates and confidence intervals using scatter plot glyphs.

We can draw a set of vertical point plots grouped by a categorical variable as follows:-

In [215…
```
sns.pointplot(x="time", y="total_bill", data=tips)

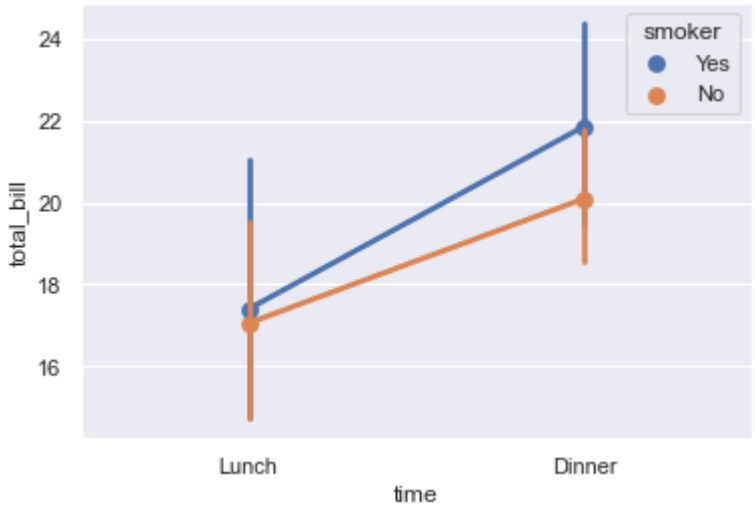
plt.show()
```



We can draw a set of vertical points with nested grouping by a two variables as follows:-

In [217…
```
sns.pointplot(x="time", y="total_bill", hue="smoker", data=tips)

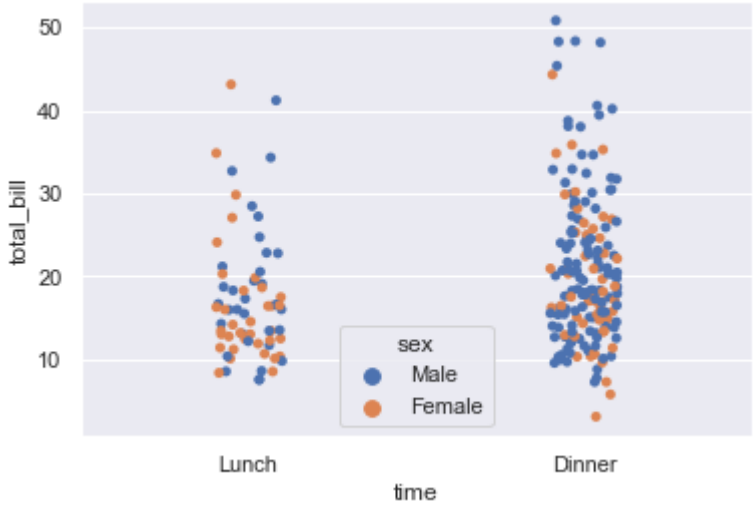
plt.show()
```



In [185…

```
sns.stripplot(x="time", y="total_bill", hue="sex", data=tips)

plt.show()
```



26. Linear relationships with Seaborn

Many datasets contain multiple quantitative variables, and the goal of an analysis is often to relate those variables to each other. We can use statistical models to estimate a simple relationship between two sets of observations. These are termed as regression models.

While building the regression models, we often check for multicollinearity, where we had to see the correlation between all the combinations of continuous variables and will take necessary action to remove multicollinearity if exists.

There are two main functions in Seaborn to visualize a linear relationship determined through regression. These functions are **regplot()** and **lmplot()**. There is a third function **residplot()** that plot the residuals of a linear regression model.

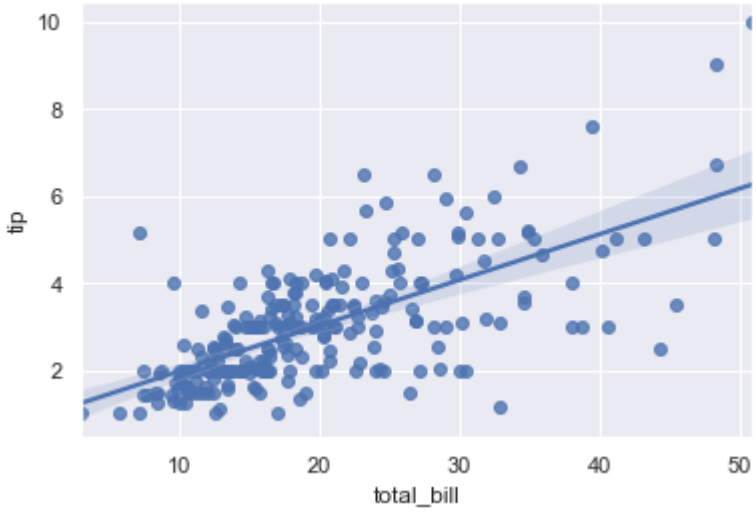
27. Seaborn – Reg plot

The function regplot() plot data and a linear regression model fit. It draws a scatterplot of two variables, x and y, and then fit the regression model y ~ x and plot the resulting regression line and a 95% confidence interval for that regression.

We can draw the scatterplot and regression line using regplot() as follows:-

In [186…
```
sns.regplot(x="total_bill", y="tip", data=tips)

plt.show()
```



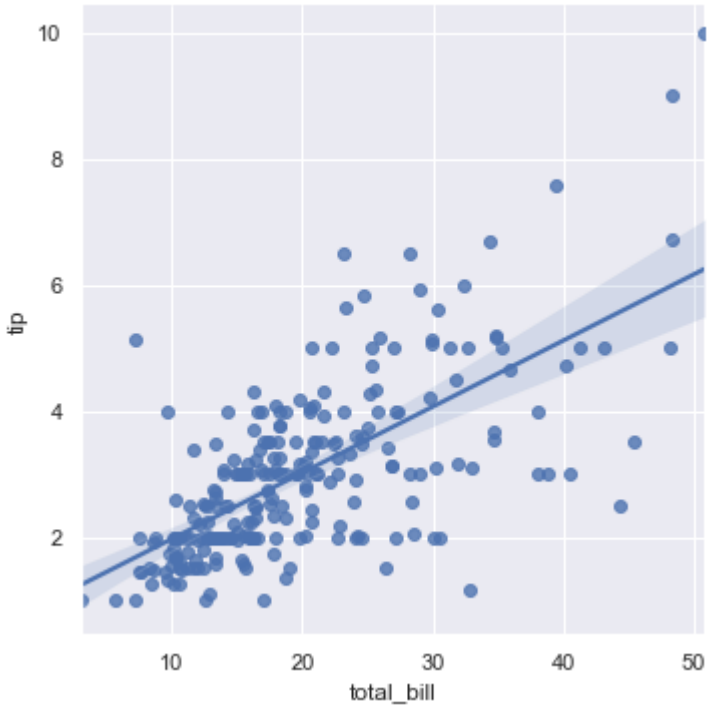
28. Seaborn – Lm plot

The function **lmplot()** plot data and regression model fits across a FacetGrid. This function combines regplot() and FacetGrid. It is intended as a convenient interface to fit regression models across conditional subsets of a dataset.

We can plot a simple linear relationship between two variables using lmplot() as follows:-

In [187…
```
sns.lmplot(x="total_bill", y="tip", data=tips)

plt.show()
```



The **regplot()** and **lmplot()** functions are closely related, but the former is an axes-level function while the latter is a figure-level function that combines **regplot()** and **FacetGrid**.

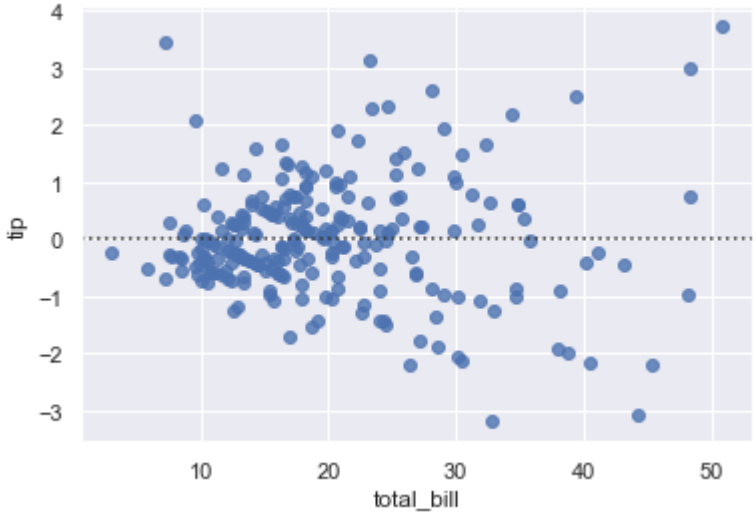
29. Seaborn – Resid plot

The function **residplot()** plot the residuals of a linear regression. We can plot the residuals as follows:-

In [188…

```
sns.residplot(x="total_bill", y="tip", data=tips)

plt.show()
```



30. Matrix plots with Seaborn

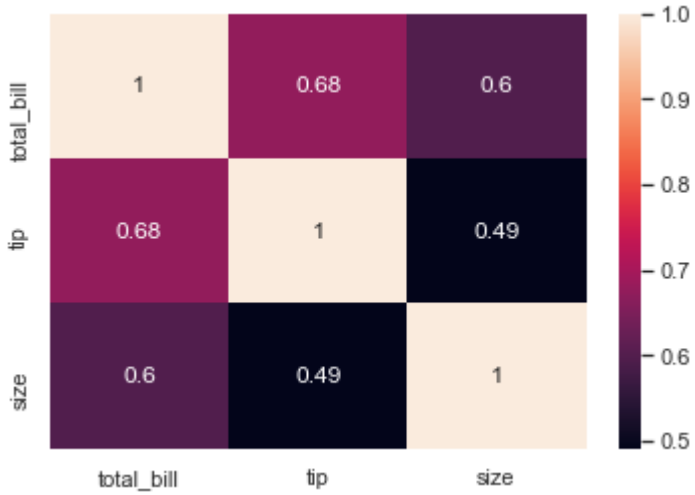
There are two functions which enable us to plot data in the form of a matrix. These are heat map and cluster map.

31. Seaborn – Heat map

Seaborn heatmap() function plot rectangular data as a color-encoded matrix.

We can plot a heatmap for a numpy array as follows:-

In [328…
```
sns.heatmap(tips.corr(),annot=True)
plt.figure(figsize=(10,5))
plt.show()
```




```
<Figure size 720x360 with 0 Axes>
```

In [333…
```
print(tips.corr())
```

```
            total_bill       tip      size
total_bill    1.000000  0.675734  0.598315
tip           0.675734  1.000000  0.489299
size          0.598315  0.489299  1.000000
```

32. Seaborn – Cluster map

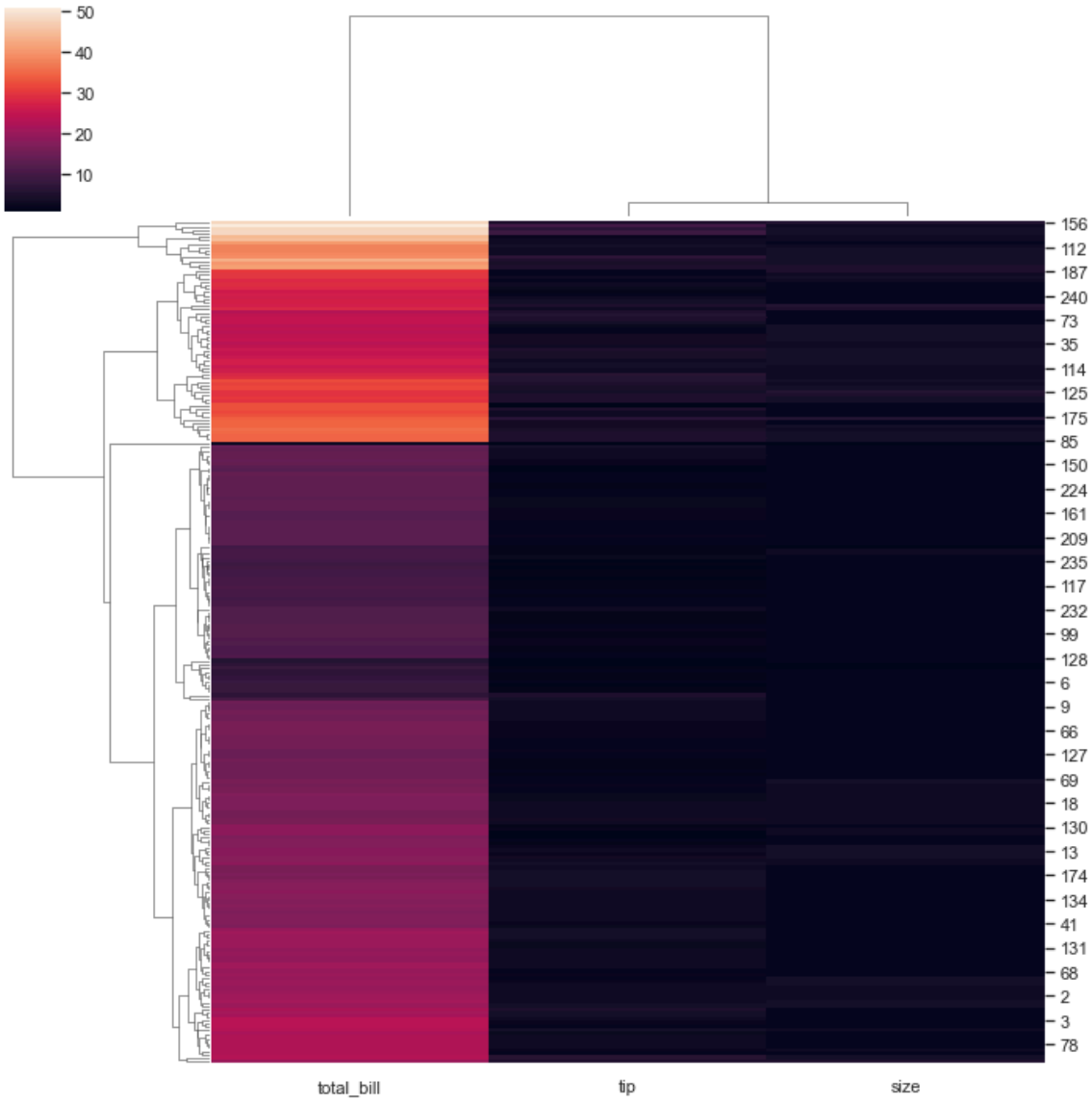
Seaborn **clustermap()** function plot a matrix dataset as a hierarchically-clustered heatmap.

We can plot a clustered heatmap of tips dataset as follows:-

In [349…
```
df1 = tips[['total_bill', 'tip', 'size']]
```

In [337…
```
sns.clustermap(df1)

plt.show()
```



33. Multi-plot grids with Seaborn

When exploring medium-dimensional data, a useful approach is to draw multiple instances of the same plot on different subsets of the dataset. This technique is sometimes called either “lattice”, or “trellis” plotting, and it is related to the idea of drawing multiple instances of the same plot. It allows us to quickly extract a large amount of information about complex data.

Seaborn provides three types of grids – **facet grid**, **pair grid** and **joint grid** to draw multiple instances of the same plot.

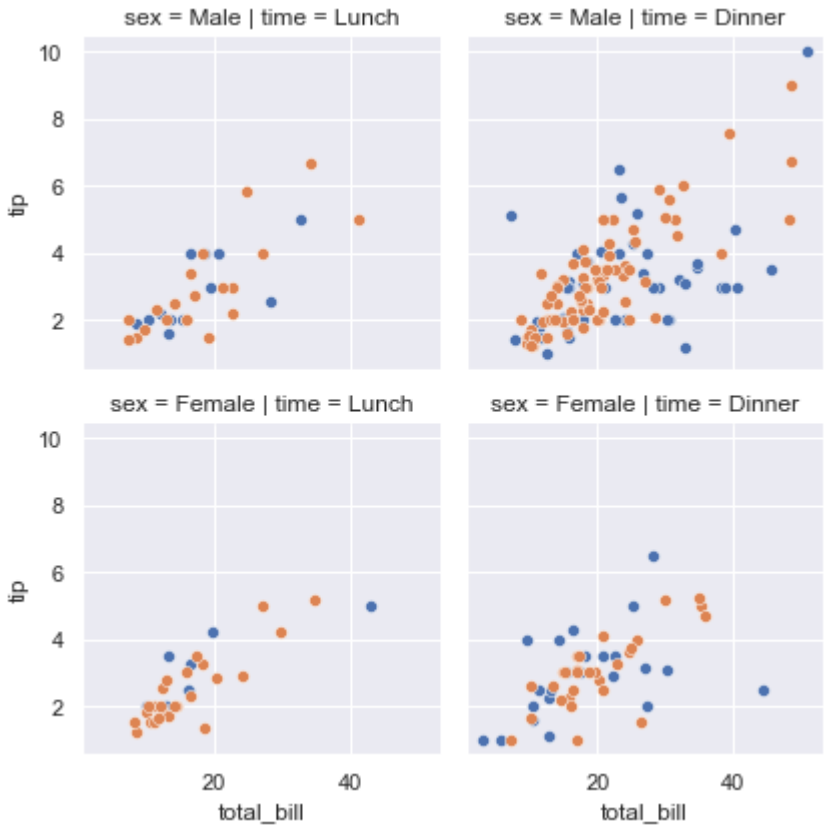
34. Seaborn – Facet Grid

Seaborn **Facetgrid()** function enable us to draw multi-plot grid for plotting conditional relationships.

First of all, we initialize a 2x2 grid of facets using the tips dataset as follows:-

In [361…
```
g = sns.FacetGrid(tips, col="time",  row="sex", hue='smoker')
g.map(sns.scatterplot, "total_bill", "tip")
```

Out[361]:
```
<seaborn.axisgrid.FacetGrid at 0x25ca14b8970>
```



Initializing the grid like this sets up the matplotlib figure and axes, but doesn't draw anything on them.

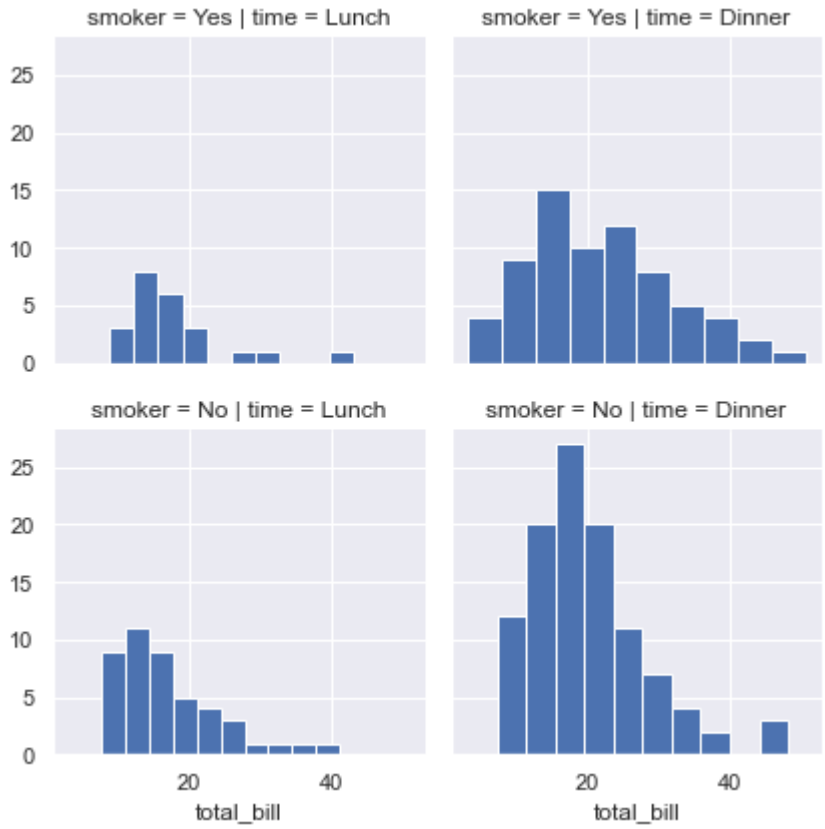
The main approach for visualizing data on this grid is with the **FacetGrid.map() method**. We need to provide it with a plotting function and the name(s) of variable(s) in the dataframe to plot.

We can draw a univariate plot on each facet as follows:-

In [363…
```
#hue='sex'
g = sns.FacetGrid(tips, col="time",  row="smoker")

g = g.map(plt.hist, "total_bill")

plt.show()
```



35. Seaborn – Pair Grid

Seaborn **PairGrid()** function draws subplot grid for plotting pairwise relationships in a dataset.

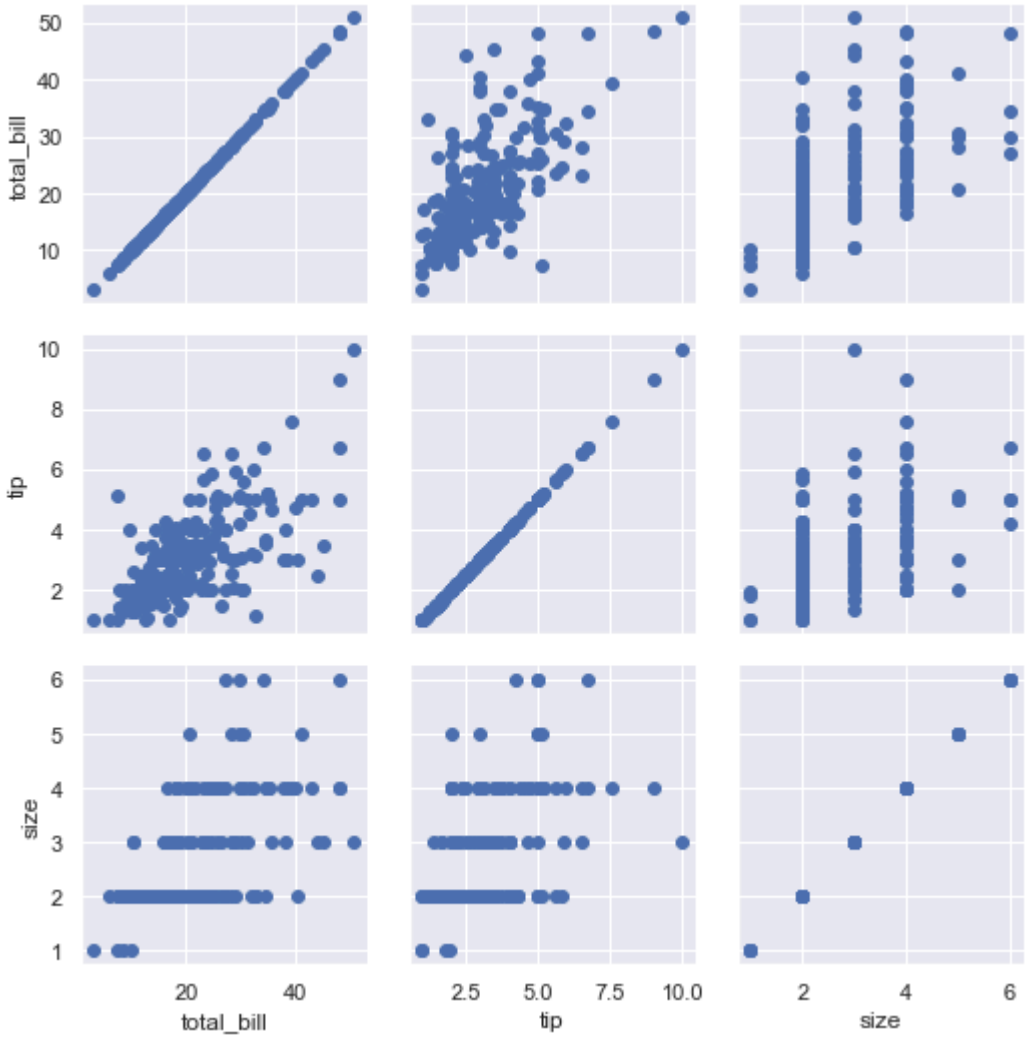
We can draw a scatterplot for each pairwise relationship using **PairGrid()** function as follows:-

In [367…

```
g = sns.PairGrid(tips)

g = g.map(plt.scatter)

plt.show()
```



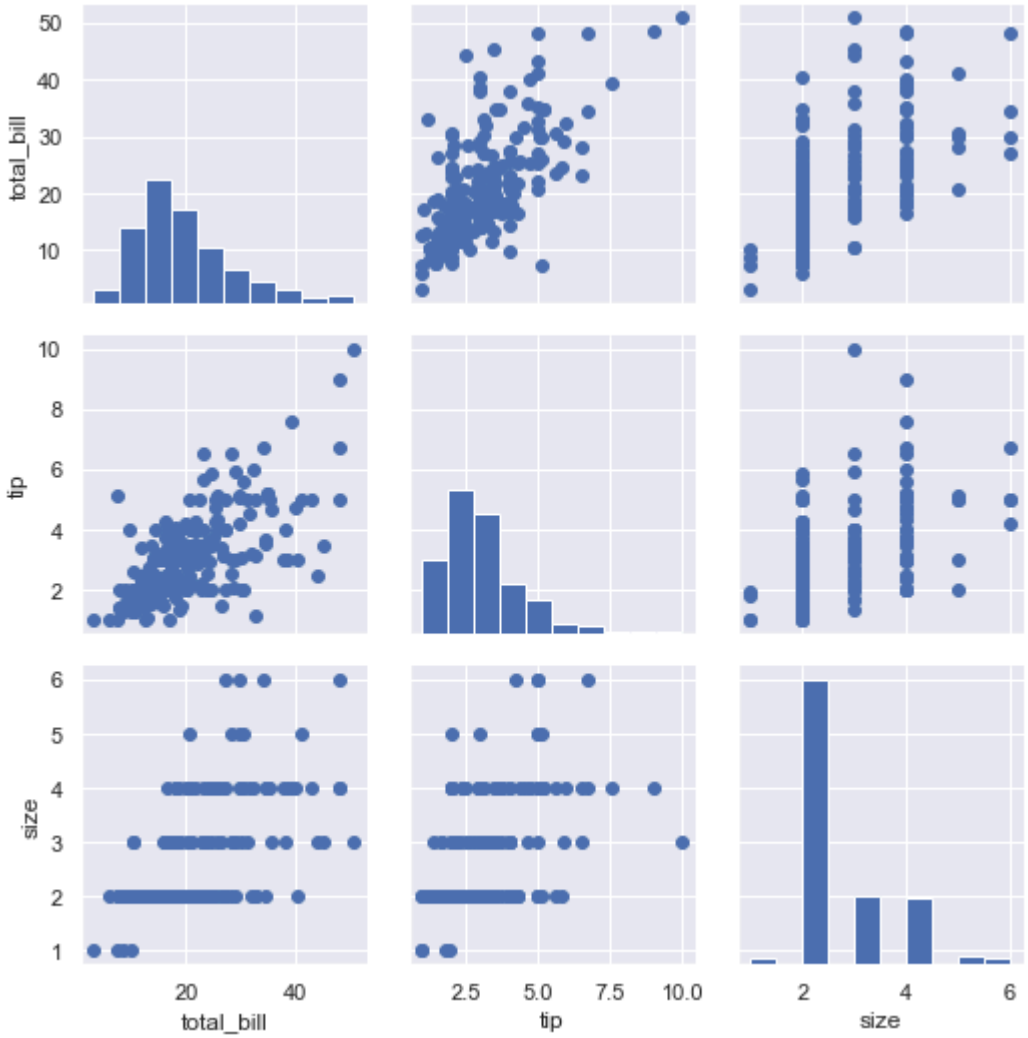
We can show a univariate distribution on the diagonal as follows:-

In [368…

```
g = sns.PairGrid(tips)

g = g.map_diag(plt.hist)

g = g.map_offdiag(plt.scatter)
```



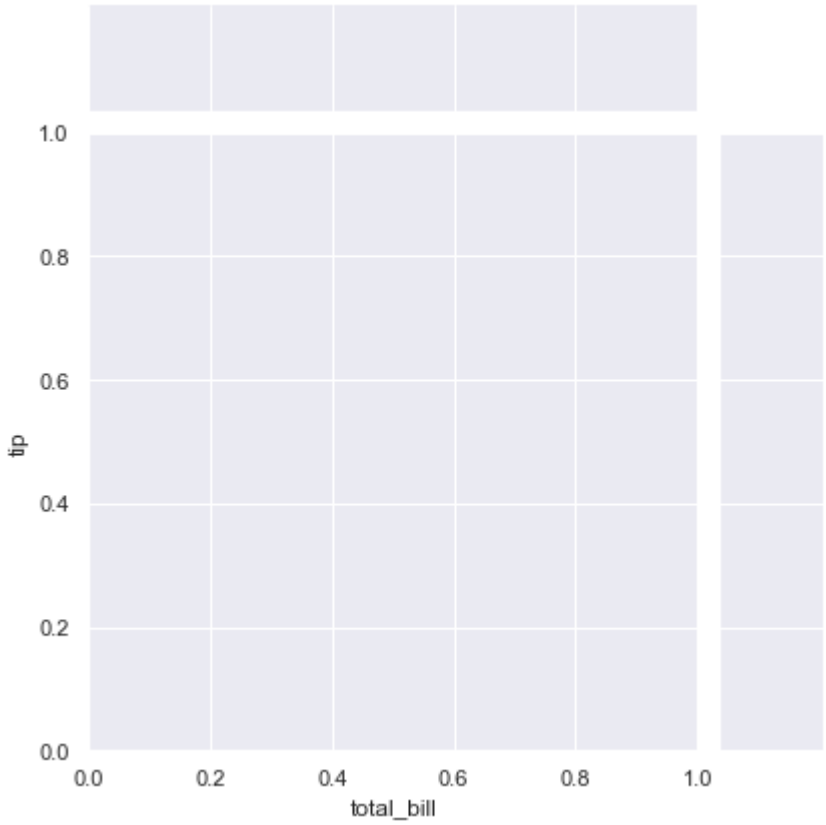
36. Seaborn – Joint Grid

Seaborn **JointGrid()** function help us to set up the grid of subplots.

We can initialize the figure but don't draw any plots onto it as follows:-

In [359…

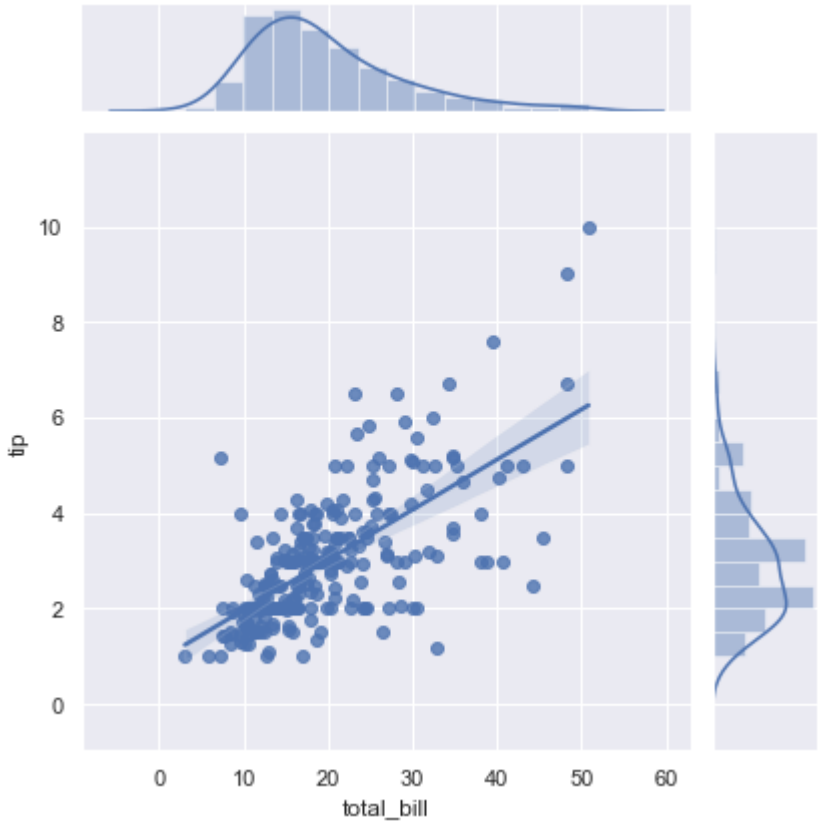
```
g = sns.JointGrid(x="total_bill", y="tip", data=tips)
```



We can add plots using default parameters as follows:-

In [347…
```
g = sns.JointGrid(x="total_bill", y="tip", data=tips)

g = g.plot(sns.regplot, sns.distplot)
```



In [ ]:
```
```